

A pipeline that notices when it's wrong

6 failures in 32 days. 5 caught on arrival, one found by hand — and not a dollar lost to any of them.

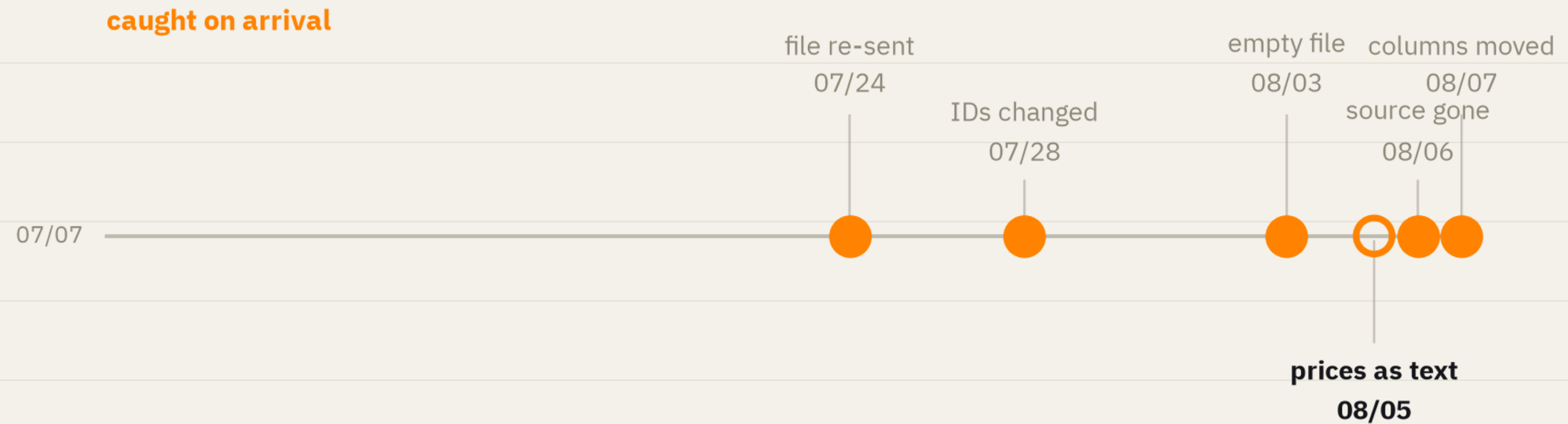
THE MONTH, AS THE PIPELINE RECORDED IT

● caught on arrival ○ found later, by hand



The feed broke 6 times in 32 days. We lost **zero dollars**.

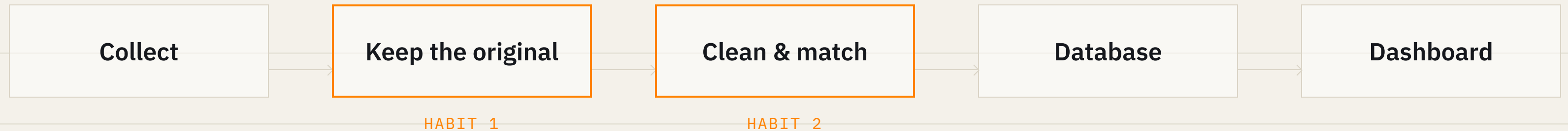
5 caught on arrival. One got through. We found it and recovered every cent.



\$1,374,672.31 in sales across the window, reconciled to the cent.

Two habits did all the protective work: keep the original, **never trust column order.**

Everything that follows traces back to these two decisions.



The database lives with the code, because our automation can't reach the campus server. Cost: it isn't a guaranteed single source of truth.

5 of 6 failures were caught the moment they arrived.

WHEN	WHAT WENT WRONG	CAUGHT BY	WHAT HAPPENED
● Jul 24	Yesterday's file sent again	Arrival check	140 duplicate rows set aside
● Jul 28	Every product ID changed	Quality flag	Mapping rebuilt; nothing lost
● Aug 3	File arrived empty	Arrival check	Logged empty; nothing invented
○ Aug 5	Prices arrived as text	Nothing — found in audit	\$56,970.09 recovered
● Aug 6	Source vanished	Arrival check	Logged failed; clean the next day
● Aug 7	Columns shuffled	Design absorbed it	Zero impact

144 rows set aside, never deleted — 140 from the re-sent file, 4 from an ordinary day.

A dollar sign turned \$57K into \$0. We got every cent back.

Three separate checks passed it. The preserved original is why recovery took minutes.

IT PASSED THREE CHECKS

- A “\$” isn't a missing value
- The parser silently gave up
- Totals matched — both sides were empty

FOUND BY

- A hand audit, weeks later
- No check fired
- Nobody saw it on the dashboard

RECOVERED

\$56,970.09
across 155 order lines, derived twice independently — then a new check added.

We attacked our own fix with a poisoned file. It caught **all five**.

A fix you haven't tested is a claim, not a fix.

```
poison fixture · executed 2026-08-08 · repo HEAD 1c4884f
fixture: C:\Users\conno\AppData\Local\Temp\poison_fixture_t4e8smht\orders_2026-08-08.csv (5 rows, every unit_price a $-prefixed string)

>>> dq.run_quality_checks(fixture, expected_date='2026-08-08')
      n_rows=5  na_unit_price=0  (isna() sees a healthy file)
      flags: schema_new_product_col;nonnumeric(unit_price=5)

>>> transform._to_float(raw.unit_price)  # the fixed parser
      ['$19.99', '$157.12', '$72.70', '$8.45', '$249.00']
      -> [19.99, 157.12, 72.7, 8.45, 249.0] (5/5 recovered)

GATE FIRED: the file reads clean to null checks but is flagged nonnumeric(unit_price=5); the parser recovers every value.
no repo state touched: fixture in OS temp, no writes to data/ or rocky_top.db
```

Executed 2026-08-08 at commit 1c4884f – a synthetic price-as-text file pushed through the **real** check and the **real** parser. Log and metadata committed.

Every product got a new ID overnight. We re-matched 76 of 80 and **flagged every doubt**.

80

products in the old catalogue

51



identical name —
certain

25



matched on evidence

4



no candidate at all

76 of 80

mapped to a new ID

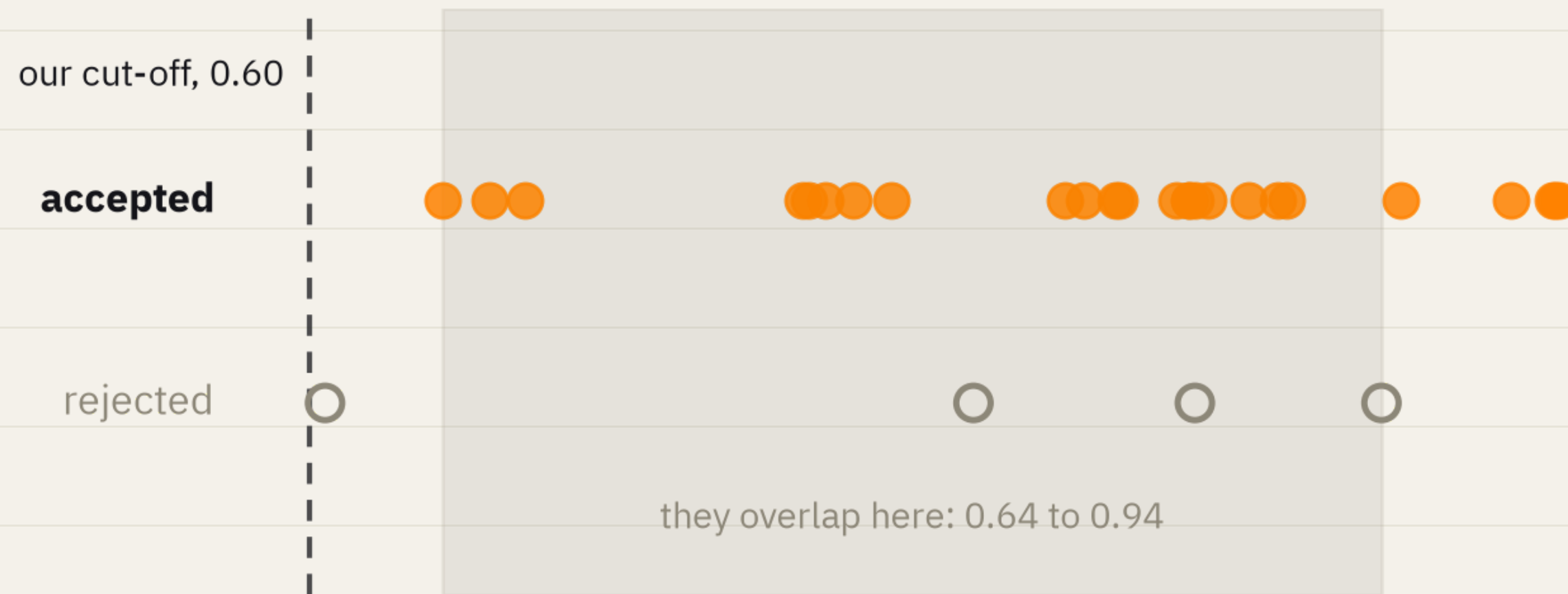
18

flagged for a human, each with a written
reason

Two thresholds, two jobs. One decides match or no match. The other decides **confident** or **check this**.

Four decoy products were planted in the data. The matcher hesitated on **exactly those four**.

EVERY CANDIDATE SCORE, ACCEPTED AGAINST REJECTED



P1077, P1078, P1079, P1080 → flagged as too close to call. **Accepted and rejected scores overlap**, so there is no clean cut – we put ours where a reviewer could still work through it.

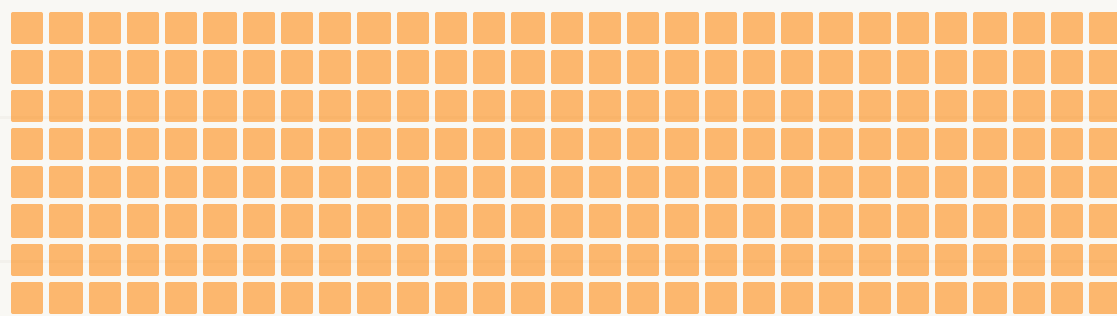
A month of sales is 1,367 rows — but only 232 independent weather readings.

Weather varies by store and day, not by product category.

232

independent weather readings

8 stores × 29 selling days



each one shared by
every category in that
store-day

1,367

rows in the sales table

store × day × category

So 1,367 rows is **not** 1,367 pieces of evidence about weather. Every claim that follows carries that caveat.

The weather archive quietly rewrote its own history. We keep **receipts** now.

5 store-days changed after we locked our numbers. Two crossed our rain line.

↓ S007 08-07 2.3mm to 0.9mm (rain to dry); S008 08-07 0.4mm to 3.7mm (dry to rain)

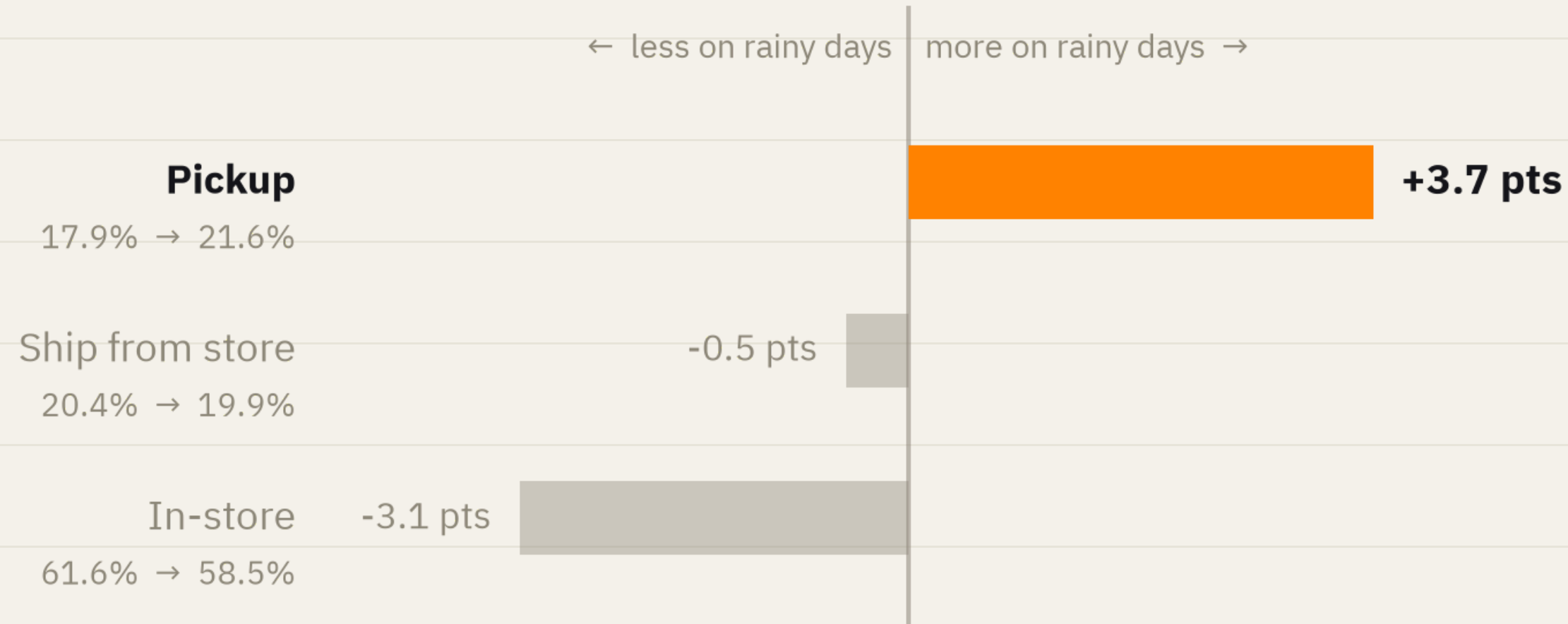
Both fall on the newest date in our window — exactly how a preliminary value behaves before it settles.

× Our cache deleted the previous pull every single run

The cache was named by date range, and the range grew daily — so each run destroyed the only record of what the archive had said before. Found in audit; pulls accumulate now.

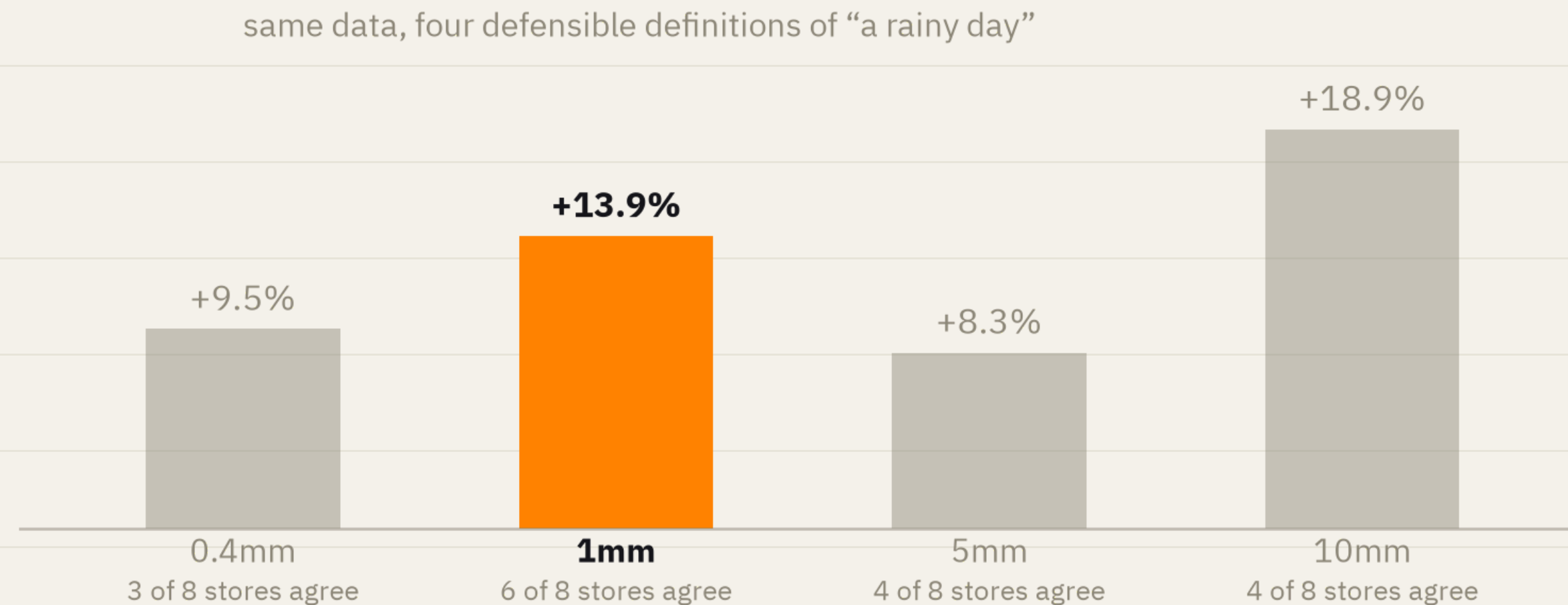
That doesn't bring anything back. Our numbers reproduce because they're pinned to a dated snapshot — **which is luck, not design.**

Rain doesn't change how much people buy. It changes **how they pick it up.**



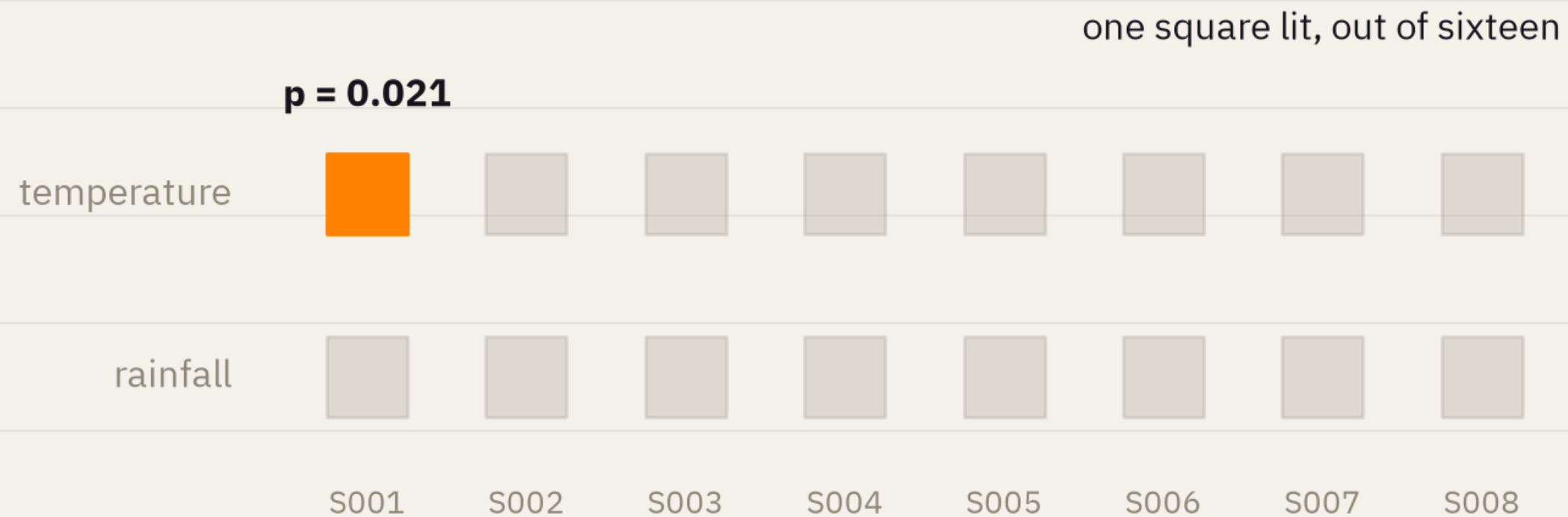
3,721 orders, 2.76σ , $p = 0.006$ – but orders in the same store-day aren't independent, so: **suggestive, not proven.**

The +14% rain revenue boost fell apart under testing — so **we don't claim it.**



And the rainiest store is also the richest — **S001: 86% rainy days, \$6,673 median day** against **S006: 14% rainy days, \$4,714 median day**. Pooling lets that impersonate a rain effect.

We ran 16 tests. One came back significant — exactly what chance predicts.



S001 vs temperature: $\rho = -0.43$, $p = 0.021$. At this sample size, reporting that one as a finding would be the fishing.

The bet worth making anyway: staff pickup when rain is forecast.

Reallocation, not spend. It needs no claim about total demand.

THE PILOT

- Two weeks, on forecast-rain days
- One associate moved to pickup staging
- At S001 (25 of 29 days) and S002 (21 of 29 days)

THE BAR, SET IN ADVANCE

- Scale only if pickup share moves 2 points
- Otherwise stop
- Wait time is the guardrail

WHY IT'S LOW-REGRET

\$0
new spend. It's a reallocation — which is what makes it worth doing on a suggestive finding.

Our safety rules live in **Python**, not in the database.

01 8 of 10 tables lost their declared keys when rebuilt

The code enforces them instead. The pipeline is correct — but the database would accept a wrong write. Fix is known.

02 14 of 18 flagged product matches flow through unblocked

The flag warns, it doesn't stop. We chose that over blocking and losing revenue — and we state the cost.

03 Our checks catch the failures we thought of

August 5th was found by hand. Nothing anomaly-based would have needed us to imagine it first.

11 AND FINALLY

\$1,374,672.31

and we can show where every dollar came from.

Including the \$57K that briefly read as zero.

P1076: the name barely matched. The other evidence carried it.

COMPONENT	SCORE	WEIGHT	CONTRIBUTION
Name similarity	0.50	0.6	0.300
Subcategory	1.00	0.2	0.200
Price closeness	0.841	0.2	0.168

Combined 0.668 – clears 0.60 by 0.068

The bar applies to the **combined** score, not the name. And because the name was weak, it was sent to review automatically – the system flagged its own thin accept.

The revenue lift at every cut-off we tested.

RAIN CUT-OFF	MEDIAN REVENUE LIFT, AND HOW MANY STORES AGREE
0.4 mm	+9.5% (3 of 8 stores)
1 mm	+13.9% (6 of 8 stores)
5 mm	+8.3% (4 of 8 stores)
10 mm	+18.9% (4 of 8 stores)

Rain is a store-day with more than 1mm of rain; lifts are medians of store-day net revenue. The headline only holds at one of these four.

What moved since we locked, and what didn't.

FIGURE	ON THESE SLIDES (LOCKED)	ON TODAY'S DATA
Pickup shift	+3.7 points	+3.4 points (z = 2.59, p = 0.009)
Rainy days at S007	S007: 12/29 rainy days locked, 11/29 today	
Rainy days at S008	S008: 5/29 rainy days locked, 6/29 today	
Revisions by the archive	5 store-days – S007 08-07 2.3mm to 0.9mm (rain to dry); S008 08-07 0.4mm to 3.7mm (dry to rain)	
Revenue, row counts and the entire product mapping did not move: 5 figures identical in the locked snapshot and today's database – asserted on every build, which fails loudly if they ever diverge.		

The poisoned-file test, in full.

```
poison fixture · executed 2026-08-08 · repo HEAD 1c4884f
fixture: C:\Users\conno\AppData\Local\Temp\poison_fixture_t4e8smht\orders_2026-08-08.csv (5 rows, every unit_price a $-prefixed string)

>>> dq.run_quality_checks(fixture, expected_date='2026-08-08')
      n_rows=5  na_unit_price=0  (isna() sees a healthy file)
      flags: schema_new_product_col;nonnumeric(unit_price=5)

>>> transform._to_float(raw.unit_price)  # the fixed parser
      ['$19.99', '$157.12', '$72.70', '$8.45', '$249.00']
      -> [19.99, 157.12, 72.7, 8.45, 249.0]  (5/5 recovered)

GATE FIRED: the file reads clean to null checks but is flagged nonnumeric(unit_price=5); the parser recovers every value.
no repo state touched: fixture in OS temp, no writes to data/ or rocky_top.db
```

Executed 2026-08-08 at commit 1c4884f. The gate flagged all 5 planted values while the old missing-value check still read clean; the parser then recovered every one of them to the cent.